SRI RAM FASHIONS DIGITAL BUSINESS MANAGEMENT SYSTEM

PROJECT DOCUMENTATION

ABSTRACT

The Sri Ram Fashions Digital Business Management System is a web-based application developed to digitalize and automate the core business operations of a textile retail enterprise. Traditional textile management relies heavily on manual record keeping, which often leads to data redundancy, calculation errors, delayed decision-making, and inefficient inventory control. This system aims to overcome these limitations by providing a centralized digital platform to manage textile inventory, customer information, sales transactions, supplier details, and GST-compliant billing processes. By integrating all business activities into a single system, the application ensures accurate data handling, improved transparency, and better operational control.

The system is designed with a user-friendly interface and secure authentication mechanisms to support both administrative and staff-level access. Real-time inventory tracking helps monitor stock availability and reduces the risk of overstocking or stock shortages. The sales and billing module allows quick processing of customer orders, automatic invoice generation with GST calculation, and efficient tracking of payment status. Additionally, customer and supplier management features help maintain long-term business relationships by storing complete transaction histories and contact details, thereby improving service quality and operational coordination.

Overall, the Sri Ram Fashions Digital Business Management System enhances business efficiency by minimizing manual effort, reducing operational costs, and improving data accuracy. The system supports informed decision-making through timely access to sales and inventory reports, enabling business owners to plan effectively and respond quickly to market demands. By adopting this digital solution, Sri Ram Fashions can achieve improved productivity, scalability, and long-term sustainability in an increasingly competitive business environment.

CHAPTER 1

INTRODUCTION

1.1 ABOUT THE PROJECT

Sri Ram Fashions is a fashion retail business management system built to handle billing, inventory, purchases, sales, and customer and supplier management in one unified platform. The application is a full-stack web system that automates GST-compliant billing, maintains accurate stock levels, and provides sales and purchase analytics to support faster business decisions.

1.2 EXISTING SYSTEM

Many retail textile businesses still depend on manual records, spreadsheets, or disconnected tools. These approaches are time-consuming, error-prone, and do not provide real-time visibility into stock, billing, and payments.

1.3 DRAWBACKS OF EXISTING SYSTEM

- Manual billing leads to calculation errors and compliance risks.
- Inventory updates are delayed, causing overstock or stock-out situations.
- Customer and supplier data is scattered and hard to track.
- Reports require manual consolidation and lack accuracy.
- No centralized audit trail for sales and purchase history.

1.4 PROPOSED SYSTEM

The proposed system provides a centralized, secure, and scalable web application for Sri Ram Fashions. It includes role-based authentication, billing with GST calculations, purchase and sales entry, inventory control, payment tracking, and reporting. The system improves accuracy, reduces manual work, and offers real-time insights into business performance.

ADVANTAGES OF PROPOSED SYSTEM

- Centralized data management for all business activities.
- GST-compliant invoice generation and print-ready templates.
- Real-time stock updates with low-stock alerts.
- Role-based access control for admin and staff.
- Sales, purchase, and stock analytics with export options.
- Faster billing and improved customer service.

CHAPTER 2

SYSTEM ANALYSIS

2.1 IDENTIFICATION OF NEED

Textile retail operations require quick billing, accurate stock control, GST compliance, and strong customer and supplier tracking. Manual methods cannot keep up with the volume and complexity of day-to-day transactions, making a digital business management system essential.

2.2 FEASIBILITY STUDY

- Technical Feasibility
- Economic Feasibility
- Operational Feasibility
- Legal and Regulatory Feasibility
- Social Feasibility

2.2.1 ECONOMIC FEASIBILITY

- Open-source technologies reduce licensing costs.
- Automation reduces manual labor and billing errors.
- Accurate inventory management reduces waste and loss.

2.2.2 OPERATIONAL FEASIBILITY

- Simple user interfaces reduce training time.
- Role-based access supports real-world store operations.
- Web access enables use across multiple devices in-store.

2.2.3 TECHNICAL FEASIBILITY

- MERN stack provides a proven architecture for scalability.
- REST APIs allow integration with future services.
- MongoDB offers flexible data modeling for retail records.

2.2.4 LEGAL AND REGULATORY FEASIBILITY

The system supports GST compliance with CGST, SGST, and IGST calculations. Authentication and access control protect sensitive business and customer data.

2.2.5 SOCIAL FEASIBILITY

The system reduces staff workload, improves service speed, and increases customer satisfaction through accurate billing and fast record retrieval.

2.3 SOFTWARE REQUIREMENTS SPECIFICATION

2.3.1 HARDWARE REQUIREMENTS

- Operating System: Windows 10/11, macOS, or Linux
- Processor: Intel i3 / Ryzen 3 or above
- RAM: 8 GB minimum (16 GB recommended)
- Storage: 20 GB free space
- Internet: Stable broadband connection

2.3.2 SOFTWARE REQUIREMENTS

- Node.js 18+ and npm
- MongoDB (local or Atlas)
- Modern browser (Chrome, Edge, or Firefox)
- Code Editor: Visual Studio Code
- Git for version control

2.4 TECHNOLOGY STACK (CORRECTED)

- Frontend: React.js + Vite (SPA)
- Styling: CSS (index.css, App.css, BillTemplate.css)
- State Management: Redux Toolkit (auth, bills, products, settings)
- Routing: React Router DOM (protected routes)
- Animations: Framer Motion
- Backend: Node.js + Express.js (REST APIs)
- Database: MongoDB Atlas (cloud)
- ODM: Mongoose
- Authentication: JWT + bcrypt
- PDF Generation: jsPDF utilities
- Excel Export: exceljs/xlsx utilities
- Email: Nodemailer and Resend integration
- AI: AI service integration for analytics
- Deployment: Render (render.yaml)

CHAPTER 3

SYSTEM DESIGN

3.1 INPUT DESIGN

Input forms are designed with validation to minimize errors. Key inputs include product details, customer data, supplier data, purchase entries, sales entries, and billing line items with GST rates and quantities.

3.2 OUTPUT DESIGN

Outputs include dashboards, GST-compliant invoices, sales and purchase reports, stock reports, and payment summaries. Reports can be exported to PDF and Excel formats.

3.3 MODULE DESIGN (ACTUAL SYSTEM MODULES)

- Homepage and Landing Page: Public-facing branding and navigation.
- Authentication (Login and Register): JWT-based login with bcrypt password hashing.
- Dashboard: Business overview with charts and key metrics.
- Billing and Invoicing: Create sales bills, GST calculation, invoice print.
- Purchase Entry: Record purchases from suppliers with tax details.
- Product and Item Management: Manage items with HSN, price, category, GST.
- Customer Management: Store customer details and transaction history.
- Supplier Management: Store supplier details and purchase history.
- Inventory and Stock: Track stock movement and low-stock alerts.
- Sales and Purchase Payments: Track payments received and paid.
- Reports (Sales and Stock): Generate reports with filters and export.
- Settings: Company info, bill template, tax configuration.
- Email Service: Automated billing emails and reminders.
- AI Analytics: Business insights and recommendations.

3.4 MODULE PRIORITY FOR DOCUMENTATION

- 1. Authentication and User Management - entry point, JWT auth, role control.
- 2. Billing and Invoice Module - core business logic with GST and invoice print.
- 3. Product and Item Management - foundation for billing and inventory.
- 4. Inventory and Stock Management - stock in and stock out tracking.
- 5. Purchase Entry Module - supplier purchases and stock updates.
- 6. Sales Entry and Reports - revenue insights and analytics.
- 7. Customer and Supplier Management - manage business relationships.
- 8. Payment Management - track receivables and payables.
- 9. Email and AI Services - automated communication and insights.
- 10. Settings and Dashboard - configuration and business overview.

3.5 DATA MODELS

- User, Product, Category, HSN, Bill, BillItem, Customer, Supplier
- SalesEntry, PurchaseEntry, Payment, StockMovement, Settings

3.6 DATA FLOW DIAGRAM

- Level 0 DFD: User interacts with Sri Ram Fashions System.
- Level 1 DFD: Billing, Inventory, Purchase, Sales, Reports, Settings modules.
- Level 2 DFD: Detailed flow for Billing and GST calculation.

CHAPTER 4

IMPLEMENTATION

4.1 SYSTEM IMPLEMENTATION

The application follows a 3-tier architecture with React frontend, Express backend, and MongoDB data storage. API endpoints are protected with JWT tokens. Redux Toolkit manages frontend state, and jsPDF utilities generate GST-compliant invoices.

4.2 STANDARDIZATION OF THE CODING

The codebase follows consistent naming conventions, modular components, and API route separation. ESLint enforces formatting standards and common utilities reduce duplication.

4.3 ERROR HANDLING

Frontend form validation prevents invalid input. Backend routes use try-catch blocks with HTTP status codes. Errors are logged and returned in a safe, user-friendly format.

4.4 KEY CODE SNIPPETS (SELECTED)

User Model (backend/src/models/User.js)

```
const userSchema = new mongoose.Schema({

    name: { type: String, required: true, trim: true },

    email: { type: String, required: true, unique: true },

    password: { type: String, required: true },

    role: { type: String, enum: ['admin','staff'], default: 'staff' }

}, { timestamps: true });
```

Product Model (backend/src/models/Product.js)

```
const productSchema = new mongoose.Schema({

    name: { type: String, required: true },

    sku: { type: String, required: true, unique: true },

    mrp: { type: Number, required: true },

    sellingPrice: { type: Number, required: true },

    hsn: String,

    gstRate: { type: Number, default: 12 }

}, { timestamps: true });
```

Bill Model (backend/src/models/Bill.js)

```
const billSchema = new mongoose.Schema({

    billNumber: { type: String, required: true, unique: true },

    customer: { name: { type: String, required: true } },

    items: [billItemSchema],

    subtotal: { type: Number, required: true },

    totalTax: Number,

    grandTotal: { type: Number, required: true }

}, { timestamps: true });
```

Auth Route (backend/src/routes/auth.js)

```
router.post('/login', async (req, res) => {

    const { email, password } = req.body;

    const user = await User.findOne({ email, isActive: true });

    const isMatch = await bcrypt.compare(password, user.password);

    const token = jwt.sign({ userId: user._id }, JWT_SECRET, { expiresIn: '7d' });

    res.json({ success: true, token, user: { id: user._id, role: user.role } });

});
```

Create Bill Flow (backend/src/routes/bills.js)

```
router.post('/', async (req, res) => {

    const { customer, items, discount = 0 } = req.body;

    let subtotal = 0; let totalTax = 0;

    for (const item of items) {

        const product = await Product.findById(item.productId);

        const itemSubtotal = item.price * item.quantity;

        const gstAmount = (itemSubtotal * product.gstRate) / 100;

        subtotal += itemSubtotal; totalTax += gstAmount;

        product.stock -= item.quantity; await product.save();

    }

    const discountAmount = (subtotal * discount) / 100;
```

```
    const grandTotal = (subtotal - discountAmount) + totalTax;

    const bill = new Bill({ billNumber: generateBillNumber(), customer, subtotal,
totalTax, grandTotal });

    await bill.save();

    res.status(201).json({ success: true, data: bill });

});
```

Server Setup (backend/src/index.js)

```
const app = express();

app.use(cors(corsOptions));

app.use(express.json());

app.use('/api/auth', authRoutes);

app.use('/api/bills', billRoutes);

app.use('/api/products', productRoutes);

mongoose.connect(MONGODB_URI).then(() => console.log("Connected"));
```

Routing (frontend-new/src/App.jsx)

```
const ProtectedRoute = ({ children }) => {

  const { isAuthenticated } = useSelector((state) => state.auth);

  return isAuthenticated ? children : <Navigate to="/login" replace />;

};

...

<Route path="/dashboard" element={<ProtectedRoute><MainLayout /></ProtectedRoute>}>

  <Route path="billing" element={<BillingPage />} />

</Route>
```

API Service (frontend-new/src/services/api.js)

```
const api = axios.create({ baseURL: API_URL, withCredentials: true });

api.interceptors.request.use((config) => {

    const token = localStorage.getItem("token");

    if (token) config.headers.Authorization = `Bearer ${token}`;

    return config;
```

```
});
```

Auth Slice (frontend-new/src/store/slices/authSlice.js)

```
export const login = createAsyncThunk(
    'auth/login',
    async ({ email, password }, { rejectWithValue }) => {
        const response = await authAPI.login(email, password);
        const { token, user } = response.data;
        localStorage.setItem("token", token);
        return { token, user };
    }
);
```

Invoice PDF Generator (frontend-new/src/utils/invoiceGenerator.js)

```
export const generateInvoicePDF = (bill, settings = {}) => {
    const pdf = new jsPDF({ orientation: "portrait", unit: "mm", format: "a4" });
    const totalAmt = Math.round((bill.subtotal - bill.discountAmount) + bill.totalTax);
    pdf.text(`TOTAL: ${totalAmt}`, 180, 280, { align: "right" });
    return pdf.save();
};
```

Email Service (backend/src/services/emailService.js)

```
const sendEmail = async (to, subject, html, attachments = []) => {
  if (resend) {
    return await resend.emails.send({ from, to, subject, html, attachments });
  }
  if (transporter) {
    return await transporter.sendMail({ from, to, subject, html, attachments });
  }
};
```

Payment Model (backend/src/models/Payment.js)

```
const paymentSchema = new mongoose.Schema({
```

```
    type: { type: String, enum: ['sales','purchase'], required: true },

    companyName: { type: String, required: true },

    amount: { type: Number, required: true },

    paymentType: { type: String, default: "cash" }

}, { timestamps: true });
```

Reports Route (backend/src/routes/reports.js)

```
router.get('/sales-summary', async (req, res) => {

    const summary = await Bill.aggregate([{ $group: { _id: null, totalRevenue: { $sum:
"$grandTotal" } } }]);

    res.json({ success: true, data: summary[0] || {} });

});
```

CHAPTER 5

TESTING AND RESULTS

5.1 TESTING

Testing ensures the system meets functional requirements and produces accurate billing, stock updates, and reports. Unit testing checks business logic, integration testing validates API and database interactions, and validation testing confirms user requirements.

5.1.1 UNIT TESTING

- Billing form validation for required fields and GST calculation.
- Product creation validation for price and stock.
- Authentication validation for login and password hashing.

5.1.2 INTEGRATION TESTING

- Create bill and verify inventory stock reduction.
- Purchase entry adds stock and reflects in inventory.
- Reports return correct aggregated totals.

5.1.3 VALIDATION TESTING

- GST invoice totals match manual calculations.
- Role-based access prevents unauthorized actions.
- Dashboard metrics match sales and purchase data.

CHAPTER 6

CONCLUSION AND FUTURE ENHANCEMENTS

6.1 CONCLUSION

The Sri Ram Fashions Digital Business Management System streamlines retail operations by automating billing, inventory, and reporting. It improves accuracy, reduces manual effort, and provides real-time business insights.

6.2 FUTURE ENHANCEMENT

- Mobile app deployment for shop floor usage.
- Barcode or QR-based inventory scanning.
- Multi-branch support with centralized reporting.
- Advanced AI-based demand forecasting.
- WhatsApp and SMS notification integration.

CHAPTER 7

REFERENCES

Book References

- Clean Code by Robert C. Martin
- Designing Interfaces by Jenifer Tidwell

Web References

- React documentation
- Express.js documentation
- MongoDB documentation
- Mongoose documentation
- Vite documentation

APPENDICES

APPENDIX A: MODULE SCREENSHOTS (PLACEHOLDERS)

- Figure A.1 Login Page
- Figure A.2 Dashboard
- Figure A.3 Billing and Invoice
- Figure A.4 Inventory
- Figure A.5 Items Master
- Figure A.6 Customer Management
- Figure A.7 Supplier Management
- Figure A.8 Sales Reports
- Figure A.9 Stock Reports
- Figure A.10 Settings

APPENDIX B: DFD AND DATABASE DIAGRAMS (PLACEHOLDERS)

- Figure B.1 Level 0 DFD
- Figure B.2 Level 1 DFD
- Figure B.3 ER Diagram or Schema Overview

APPENDIX C: ADDITIONAL PAGES (PLACEHOLDERS)

Appendix Page C.1: Placeholder for screenshots, DFD, ER diagrams, or extra content.

Appendix Page C.2: Placeholder for screenshots, DFD, ER diagrams, or extra content.

Appendix Page C.3: Placeholder for screenshots, DFD, ER diagrams, or extra content.

Appendix Page C.4: Placeholder for screenshots, DFD, ER diagrams, or extra content.

Appendix Page C.5: Placeholder for screenshots, DFD, ER diagrams, or extra content.

Appendix Page C.6: Placeholder for screenshots, DFD, ER diagrams, or extra content.

Appendix Page C.7: Placeholder for screenshots, DFD, ER diagrams, or extra content.

Appendix Page C.8: Placeholder for screenshots, DFD, ER diagrams, or extra content.

Appendix Page C.9: Placeholder for screenshots, DFD, ER diagrams, or extra content.

Appendix Page C.10: Placeholder for screenshots, DFD, ER diagrams, or extra content.

Appendix Page C.11: Placeholder for screenshots, DFD, ER diagrams, or extra content.

Appendix Page C.12: Placeholder for screenshots, DFD, ER diagrams, or extra content.

Appendix Page C.13: Placeholder for screenshots, DFD, ER diagrams, or extra content.

Appendix Page C.14: Placeholder for screenshots, DFD, ER diagrams, or extra content.

Appendix Page C.15: Placeholder for screenshots, DFD, ER diagrams, or extra content.

Appendix Page C.16: Placeholder for screenshots, DFD, ER diagrams, or extra content.

Appendix Page C.17: Placeholder for screenshots, DFD, ER diagrams, or extra content.

Appendix Page C.18: Placeholder for screenshots, DFD, ER diagrams, or extra content.

Appendix Page C.19: Placeholder for screenshots, DFD, ER diagrams, or extra content.

Appendix Page C.20: Placeholder for screenshots, DFD, ER diagrams, or extra content.

Appendix Page C.21: Placeholder for screenshots, DFD, ER diagrams, or extra content.

Appendix Page C.22: Placeholder for screenshots, DFD, ER diagrams, or extra content.

Appendix Page C.23: Placeholder for screenshots, DFD, ER diagrams, or extra content.

Appendix Page C.24: Placeholder for screenshots, DFD, ER diagrams, or extra content.

Appendix Page C.25: Placeholder for screenshots, DFD, ER diagrams, or extra content.

Appendix Page C.26: Placeholder for screenshots, DFD, ER diagrams, or extra content.

Appendix Page C.27: Placeholder for screenshots, DFD, ER diagrams, or extra content.

Appendix Page C.28: Placeholder for screenshots, DFD, ER diagrams, or extra content.

Appendix Page C.29: Placeholder for screenshots, DFD, ER diagrams, or extra content.

Appendix Page C.30: Placeholder for screenshots, DFD, ER diagrams, or extra content.

Appendix Page C.31: Placeholder for screenshots, DFD, ER diagrams, or extra content.

Appendix Page C.32: Placeholder for screenshots, DFD, ER diagrams, or extra content.

Appendix Page C.33: Placeholder for screenshots, DFD, ER diagrams, or extra content.

Appendix Page C.34: Placeholder for screenshots, DFD, ER diagrams, or extra content.

Appendix Page C.35: Placeholder for screenshots, DFD, ER diagrams, or extra content.

Appendix Page C.36: Placeholder for screenshots, DFD, ER diagrams, or extra content.

Appendix Page C.37: Placeholder for screenshots, DFD, ER diagrams, or extra content.

Appendix Page C.38: Placeholder for screenshots, DFD, ER diagrams, or extra content.

Appendix Page C.39: Placeholder for screenshots, DFD, ER diagrams, or extra content.

Appendix Page C.40: Placeholder for screenshots, DFD, ER diagrams, or extra content.

Appendix Page C.41: Placeholder for screenshots, DFD, ER diagrams, or extra content.

Appendix Page C.42: Placeholder for screenshots, DFD, ER diagrams, or extra content.

Appendix Page C.43: Placeholder for screenshots, DFD, ER diagrams, or extra content.

Appendix Page C.44: Placeholder for screenshots, DFD, ER diagrams, or extra content.

Appendix Page C.45: Placeholder for screenshots, DFD, ER diagrams, or extra content.